Special functions in Mathilda: a four-engine accuracy and speed shootout

Mathilda benchmark suite

2026-10-17

Abstract

We benchmark every numerically-evaluable special function in **Mathilda** (56 heads) against three external systems — **Python** (SciPy for machine precision, mpmath for arbitrary precision), **Maxima**, and **Mathematica** (via `wolframscript`) — across three regimes: machine-real, complex, and arbitrary precision (50 digits). Two things are measured: *speed* (per-call scalar latency in every regime, plus machine-real vectorized throughput over a 10^6-element buffer), and *accuracy*, defined *oracle-free* as the number of leading significant digits to which the engines mutually agree, with any engine that falls out of the consensus flagged. The sweep is self-contained — it does not touch the weekly `run_all.py` gap gate — and regenerates this document's tables from one results JSON.

1 Method

Engines and versions.

- **Mathilda** — 0.446 (Mathilda 0.446 (GCC 16.1.0, GMP 6.3.0, MPFR 4.2.2, FLINT 3.6.0, ECM 7.0.7, Raylib 5.5, Accelerate, Readline)).
- **Mathematica** — 14. (14.0.0 for Mac OS X x86 (64-bit) (December 13, 2023)), driven by `wolframscript -file`.
- **Maxima** — Maxima 5.49.0.
- **Python** — scipy 1.18.1, mpmath 1.3.0, numpy 2.5.2. SciPy's `scipy.special` (compiled C) backs the machine-real and complex columns; mpmath backs arbitrary precision (SciPy is double-only) and is also the fallback for the handful of heads SciPy lacks.

Platform: macOS-15.7.4-x86_64-i386-64bit-Mach-O.

The three regimes. Each head is exercised at (MR) machine-real arguments, computed to IEEE double; (C) complex machine arguments; and (AP) exact-rational arguments evaluated to 50 digits. In Mathilda these correspond to the `Gamma[2.5]`, `Gamma[2.5+1.3 I]`, and `N[Gamma[5/2], 50]` dispatch paths respectively. Arguments are drawn deterministically from each head's real principal domain so that timing measures the kernel, not a decline-to-fallback path. Arbitrary precision is sampled on the real axis.

Accuracy: agreement, not an oracle. No engine is designated the source of truth. For each sample point we collect the value from every engine that returns a number, take the componentwise median as the consensus, and report the number of leading significant digits the engines mutually share ($-\log_{10}$ of the largest pairwise relative spread). An engine whose value deviates from the consensus by more than the regime tolerance — while the others agree closely — is *flagged*. Four independent implementations agreeing to ≈ 15 digits in machine precision (or to 50 in AP) is strong evidence of correctness; a lone dissenter is either a genuine branch/convention

difference (documented) or a bug. This agreement check doubled as the cross-engine mapping validator while the benchmark was built.

Speed. Latency is timed *inside* each engine (Mathilda/Mathematica `AbsoluteTiming`, Maxima `elapsed_real_time`, Python `perf_counter`) so process start-up is charged once, not per call. Each engine evaluates the head over a block of *distinct* arguments and we report the minimum per-call time over several such blocks: distinct arguments mean no engine can serve a memoised result (Mathematica's `N` cache and Mathilda's evaluation memo would otherwise measure the cache, not the arithmetic), and the single pass per block avoids re-evaluating identical work. `AbsoluteTiming` is used rather than `Timing`, which sums CPU time across threads.

Three further controls make the figure the *kernel's* cost.

- **Machine literals.** Machine arguments are written with a trailing backtick (`1.2345678901234567‘`). A 17-digit literal without the backtick is read as *arbitrary* precision by both Mathilda and Mathematica, so the plain form would route part of the machine sample through the MPFR paths.
- **Net latency.** Each block is timed twice: once calling the head, and once as a *null loop* with the same `Do`/`Part` (Python: `for`) loop and no call. Neither loop is wrapped in `Quiet`. The reported latency is $\min_b(\text{raw}) - \min_b(\text{null})$ (512 points per block, 5 blocks).
- **Cache defence.** `ClearSystemCache[]` is called before every timed loop. Values and timing run in separate engine processes. Python's warm-up uses its own points. A *cache canary* then re-times block 0 on points it has already seen: a repeat more than 2× faster than the first pass means the engine is serving memoised results. Such a cell is marked † and excluded from the ranking.

The vectorized-throughput table instead gives each engine its best array form: a 10^6-element packed buffer for Mathilda and a NumPy array for SciPy (Maxima and Mathematica are scalar here and omitted).

Scope. All 56 numerically-evaluable heads are covered. The exact integer/rational heads (`Fibonacci`, `LucasL`, `BernoulliB`, `EulerE`, `HarmonicNumber`, `BarnesG`, `Hyperfactorial`) are reported separately as exact arithmetic. The two symbolic-only heads (`BesselJZero`, a Hadamard-product placeholder, and `StieltjesGamma`, inert except at 0) have no numeric path and are excluded.

2 Coverage

Which engines evaluate each head in each regime. This is a result in itself: Mathilda and Mathematica cover the whole surface in all three regimes, whereas SciPy is double-only (no AP column of its own) and Maxima leaves several higher transcendental functions symbolic. Marks are ✓ supported, × not evaluated, – regime not applicable to the head.

Function	Mathilda			Mathematica			Maxima			Python		
	MR	C	AP	MR	C	AP	MR	C	AP	MR	C	AP
Gamma												
`Gamma`	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
`LogGamma`	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
`PolyGamma0`	✓	✓	✓	✓	✓	✓	✓	×	✓	✓	✓	✓
`PolyGamma2`	✓	✓	✓	✓	✓	✓	✓	×	✓	✓	✓	✓
`Beta`	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
`Pochhammer`	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zeta												
`Zeta`	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Function	Mathilda	Mathematica	Maxima	Python
HurwitzZeta	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
LerchPhi	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
PolyLog	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
Bessel/Airy				
BesselJ	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
BesselY	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ×	✓ ✓ ✓
BesselI	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
BesselK	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ×	✓ ✓ ✓
AiryAi	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
AiryBi	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
AiryAiPrime	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
AiryBiPrime	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
StruveH	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Error				
Erf	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Erfc	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Erfi	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
InverseErf	✓ – ✓	✓ – ✓	✓ – ✓	✓ – ✓
InverseErfc	✓ – ✓	✓ – ✓	✓ – ✓	✓ – ✓
FresnelC	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
FresnelS	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Elliptic				
EllipticK	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
EllipticE	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
EllipticF	✓ – ✓	✓ – ✓	✓ – ✓	✓ – ✓
EllipticEinc	✓ – ✓	✓ – ✓	✓ – ✓	✓ – ✓
EllipticPi	✓ – ✓	✓ – ✓	✓ – ✓	✓ – ✓
Orthogonal				
LegendreP	✓ ✓ ✓	✓ ✓ ✓	✓ × ✓	✓ ✓ ✓
LegendreQ	✓ ✓ ✓	✓ ✓ ✓	✓ × ✓	✓ ✓ ✓
HermiteH	✓ ✓ ✓	✓ ✓ ✓	✓ × ✓	✓ ✓ ✓
LaguerreL	✓ ✓ ✓	✓ ✓ ✓	✓ × ✓	✓ ✓ ✓
ChebyshevT	✓ ✓ ✓	✓ ✓ ✓	✓ × ✓	✓ ✓ ✓
ChebyshevU	✓ ✓ ✓	✓ ✓ ✓	✓ × ✓	✓ ✓ ✓
Integral				
SinIntegral	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
CosIntegral	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
SinhIntegral	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
CoshIntegral	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
ExpIntegralEi	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
ExpIntegralE	✓ – –	✓ – –	✓ – –	✓ – –
LogIntegral	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Hypergeometric				
Hypergeometric0F1	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Hypergeometric1F1	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Hypergeometric2F1	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
HypergeometricPFQ	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
HypergeometricU	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
WhittakerM	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
WhittakerW	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
MeijerG	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
MeijerG22	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
MeijerGlog	✓ ✓ ✓	✓ ✓ ✓	× × ×	✓ ✓ ✓
Misc				
Sinc	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
ProductLog	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓

3 Accuracy (cross-engine agreement)

Significant digits of mutual agreement per head and regime (higher is better; machine regimes are bounded by the ≈ 15.9 digits of IEEE double, AP by the 50 requested digits).

Function	MR (digits)	Complex (digits)	Arb. prec. (digits)
Gamma			
`Gamma`	15.6	14.3	50.5
`LogGamma`	14.4	13.9	50.5
`PolyGamma0`	15.0	15.3	50.3
`PolyGamma2`	14.8	15.6	50.3
`Beta`	14.6	14.1	50.4
`Pochhammer`	15.6	15.2	50.4
Zeta			
`Zeta`	15.6	15.6	50.2
`HurwitzZeta`	14.1	15.4	50.5
`LerchPhi`	15.7	15.8	50.4
`PolyLog`	15.7	15.7	50.4
Bessel/Airy			
`BesselJ`	13.1	14.4	49.8
`BesselY`	13.4	14.7	50.6
`BesselI`	15.0	15.4	50.4
`BesselK`	15.5	13.8	50.4
`AiryAi`	14.8	14.2	50.2
`AiryBi`	15.6	14.2	50.5
`AiryAiPrime`	12.8	14.4	49.9
`AiryBiPrime`	13.7	15.2	50.4
`StruveH`	13.1	15.5	50.2
Error			
`Erf`	15.6	14.7	50.5
`Erfc`	15.1	15.1	50.3
`Erfi`	14.8	15.2	50.4
`InverseErf`	15.5	–	50.2
`InverseErfc`	15.5	–	50.5
`FresnelC`	14.9	14.6	50.5
`FresnelS`	13.7	14.7	50.5
Elliptic			
`EllipticK`	15.6	15.5	50.3
`EllipticE`	15.2	15.4	50.2
`EllipticF`	15.6	–	50.6
`EllipticEinc`	15.5	–	50.2
`EllipticPi`	15.4	–	50.2
Orthogonal			
`LegendreP`	14.8	14.8	50.2
`LegendreQ`	15.1	15.3	50.6
`HermiteH`	14.5	15.6	50.1
`LaguerreL`	15.2	15.1	50.2
`ChebyshevT`	14.8	14.1	50.5
`ChebyshevU`	14.3	14.2	50.4
Integral			
`SinIntegral`	14.5	14.2	50.3
`CosIntegral`	13.1	13.8	50.5
`SinhIntegral`	15.3	14.3	50.2
`CoshIntegral`	-1.3	14.2	50.4
`ExpIntegralEi`	15.3	14.4	50.4
`ExpIntegralE`	14.2	–	–
`LogIntegral`	15.2	15.5	50.4

Function	MR	Complex	Arb. prec.
Hypergeometric			
HypergeometricOF1	14.9	15.4	50.5
Hypergeometric1F1	15.3	15.4	50.4
Hypergeometric2F1	15.8	14.7	50.3
HypergeometricPFQ	15.3	16.2	50.4
HypergeometricU	15.7	15.6	50.3
WhittakerM	15.7	15.5	50.6
WhittakerW	15.7	15.5	50.5
MeijerG	14.9	14.9	50.4
MeijerG22	14.2	14.4	50.4
MeijerGlog	12.4	12.9	50.5
Misc			
Sinc	16.2	15.7	50.3
ProductLog	15.8	15.8	50.4

3.1 Disagreements

The 1 cases where an engine fell out of the consensus. Each is a genuine branch or convention difference, not a transcription error (see the notes below the table in the discussion).

Function	Regime	Engine	Agreement (digits)
CoshIntegral	machine real	Maxima	-1.3

4 Speed: scalar latency

Per-call latency with the fastest engine's cell in bold and a trailing column giving Mathilda relative to the fastest *other* engine (a bold $1/N$ means Mathilda is $N\times$ faster than the best competitor; N means $N\times$ slower).

4.1 Machine real

Function	Mathilda	Mathematica	Maxima	Python	M/best
net per-call latency (μs), loop floor subtracted					
Gamma					
Gamma	**0.105**	0.209	1.80	0.223	**1/2.0**
LogGamma	**0.092**	0.199	2.28	0.143	**1/1.6**
PolyGamma0	**0.107**	0.408	136	0.159	**1/1.5**
PolyGamma2	**0.172**	48.5	297	9.34	**1/54.3**
Beta	**0.088**	0.568	8.61	0.855	**1/6.5**
Pochhammer	**0.086**	0.398	1.12	1.48	**1/4.6**
Zeta					
Zeta	**0.260**	32.5	8.77	1.33	**1/5.1**
HurwitzZeta	**0.223**	37.0	–	1.17	**1/5.2**
LerchPhi	**1.48**	150	–	19852	**1/101.3**
PolyLog	**0.109**	6.72	–	503	**1/61.4**
Bessel/Airy					
BesselJ	**0.188**	36.8	4.13	1.82	**1/9.7**
BesselY	**0.144**	151	3.62	2.36	**1/16.3**
BesselI	**0.117**	4.78	3.41	1.46	**1/12.4**
BesselK	**0.127**	1.13	3.78	1.41	**1/8.9**
AiryAi	**0.111**	0.225	2.07	1.72	**1/2.0**
AiryBi	**0.109**	0.240	2.07	1.77	**1/2.2**

Function	Mathilda	Mathematica	Maxima	Python	M/best
`AiryAiPrime`	**0.111**	0.229	2.09	1.73	**1/2.1**
`AiryBiPrime`	**0.111**	0.238	2.45	1.72	**1/2.1**
`StruveH`	**0.285**	331	1327	7.27	**1/25.5**
Error					
`Erf`	**0.092**	0.178	1.91	0.126	**1/1.4**
`Erfc`	**0.096**	0.191	5.00	0.153	**1/1.6**
`Erfi`	**0.092**	11.2	15.6	0.137	**1/1.5**
`InverseErf`	**0.086**	16.0	3.98	0.130	**1/1.5**
`InverseErfc`	**0.090**	17.6	13.1	0.230	**1/2.6**
`FresnelC`	**0.111**	21.6	17.4	1.39	**1/12.5**
`FresnelS`	**0.111**	21.7	17.3	1.51	**1/13.5**
Elliptic					
`EllipticK`	**0.092**	0.190	7.78	0.125	**1/1.4**
`EllipticE`	**0.092**	0.221	15.8	0.154	**1/1.7**
`EllipticF`	**0.111**	0.279	8.10	1.02	**1/2.5**
`EllipticEinc`	**0.141**	0.293	28.7	1.08	**1/2.1**
`EllipticPi`	**0.295**	13.0	79.2	1710	**1/44.1**
Orthogonal					
`LegendreP`	**0.094**	0.846	4.61	0.965	**1/9.0**
`LegendreQ`	**0.109**	19.9	4.59	2688	**1/42.0**
`HermiteH`	**0.092**	0.707	2.93	0.971	**1/7.7**
`LaguerreL`	**0.090**	0.809	2.41	0.966	**1/9.0**
`ChebyshevT`	**0.088**	1.05	3.61	0.970	**1/11.0**
`ChebyshevU`	**0.090**	1.19	3.65	0.967	**1/10.8**
Integral					
`SinIntegral`	**0.115**	88.2	13.6	1.84	**1/15.9**
`CosIntegral`	**0.129**	88.9	13.5	1.26	**1/9.8**
`SinhIntegral`	**0.105**	69.0	11.6	1.34	**1/12.7**
`CoshIntegral`	**0.107**	46.7	13.3	1.34	**1/12.5**
`ExpIntegralEi`	**0.105**	11.2	7.47	0.261	**1/2.5**
`ExpIntegralE`	**0.242**	1.30	8.07	2.10	**1/5.4**
`LogIntegral`	**0.117**	11.4	6.97	0.402	**1/3.4**
Hypergeometric					
`Hypergeometric0F1`	**0.818**	5.52	279	1.58	**1/1.9**
`Hypergeometric1F1`	**0.111**	4.04	1467	1.56	**1/14.0**
`Hypergeometric2F1`	**0.273**	14.7	1408	1.97	**1/7.2**
`HypergeometricPFQ`	**0.279**	190	1536	25.1	**1/90.0**
`HypergeometricU`	**0.092**	7.94	–	1.50	**1/16.3**
`WhittakerM`	**0.111**	10.1	–	46.6	**1/91.1**
`WhittakerW`	**0.111**	12.3	–	50.5	**1/110.2**
`MeijerG`	**15.4**	5940	–	484	**1/31.5**
`MeijerG22`	**10.3**	7146	–	1320	**1/128.4**
`MeijerGlog`	**2.99**	19478	–	1612	**1/539.1**
Misc					
`Sinc`	**0.090**	0.197	3.16	0.138	**1/1.5**
`ProductLog`	**0.223**	8.71	3.93	1.83	**1/8.2**

4.2 Complex

Function	Mathilda	Mathematica	Maxima	Python	M/best
net per-call latency (μs), loop floor subtracted					
Gamma					
`Gamma`	**0.250**	2.23	6.74	0.356	**1/1.4**
`LogGamma`	**0.217**	2.25	4.52	0.282	**1/1.3**
`PolyGamma0`	**0.180**	14.8	1.88	0.710	**1/4.0**

Function	Mathilda	Mathematica	Maxima	Python	M/best
PolyGamma2	**0.305**	231	1.84	603	**1/6.1**
Beta	**0.326**	6.79	26.8	354	**1/20.8**
Pochhammer	**0.133**	1.49	3.56	238	**1/11.2**
Zeta					
Zeta	**0.598**	59.4	21.1	392	**1/35.3**
HurwitzZeta	**0.736**	325	–	1899	**1/442.0**
LerchPhi	**1.84**	153	–	38421	**1/82.9**
PolyLog	**0.225**	116	–	1398	**1/518.4**
Bessel/Airy					
BesselJ	**0.445**	45.1	9.23	1.68	**1/3.8**
BesselY	**0.606**	70.7	12.3	2.74	**1/4.5**
BesselI	**0.381**	15.7	7.90	1.50	**1/3.9**
BesselK	**0.312**	84.1	7.64	1.62	**1/5.2**
AiryAi	**0.592**	75.0	6.07	4.92	**1/8.3**
AiryBi	**0.740**	73.4	6.03	4.83	**1/6.5**
AiryAiPrime	**0.609**	84.0	4.94	4.83	**1/7.9**
AiryBiPrime	**0.731**	82.8	6.29	4.78	**1/6.5**
StruveH	**0.229**	344	4932	153	**1/667.5**
Error					
Erf	**0.180**	44.1	24.5	0.428	**1/2.4**
Erfc	**0.185**	41.8	24.7	0.261	**1/1.4**
Erfi	**0.182**	45.0	25.5	0.242	**1/1.3**
FresnelC	**0.221**	176	31.1	2.43	**1/11.0**
FresnelS	**0.215**	152	31.3	2.20	**1/10.2**
Elliptic					
EllipticK	**0.213**	3.28	31.1	186	**1/15.4**
EllipticE	**0.215**	8.15	62.9	591	**1/37.9**
Orthogonal					
LegendreP	**0.135**	27.0	12.9	1.17	**1/8.7**
LegendreQ	**0.195**	47.9	23538	6174	**1/245.1**
HermiteH	**0.160**	1.58	10.7	88.2	**1/9.9**
LaguerreL	**0.162**	1.58	7.12	1.14	**1/7.1**
ChebyshevT	**0.160**	1.87	11.7	1.18	**1/7.4**
ChebyshevU	**0.160**	2.15	11.7	1.20	**1/7.5**
Integral					
SinIntegral	**0.869**	82.1	27.2	126	**1/31.3**
CosIntegral	**1.41**	72.4	25.9	132	**1/18.4**
SinhIntegral	**0.182**	72.8	38.6	218	**1/212.7**
CoshIntegral	**0.205**	68.8	41.5	224	**1/202.4**
ExpIntegralEi	**0.271**	18.3	30.6	90.6	**1/67.5**
LogIntegral	**0.289**	17.4	9.21	117	**1/31.9**
Hypergeometric					
Hypergeometric0F1	**0.961**	12.7	1116	1.74	**1/1.8**
Hypergeometric1F1	**0.199**	10.2	8525	1.55	**1/7.8**
Hypergeometric2F1	**1.49**	54.8	12961	2.61	**1/1.7**
HypergeometricPFQ	**0.519**	313	7774	58.0	**1/111.7**
HypergeometricU	**0.221**	8.29	–	34.2	**1/37.6**
WhittakerM	**0.225**	10.8	–	71.5	**1/48.2**
WhittakerW	**0.264**	12.8	–	85.7	**1/48.4**
MeijerG	**16.6**	6120	–	701	**1/42.3**
MeijerG22	**11.7**	7345	–	2871	**1/244.7**
MeijerGlog	**7.62**	19430	–	3574	**1/468.7**
Misc					
Sinc	**0.150**	0.299	7.65	0.194	**1/1.3**
ProductLog	**0.486**	24.1	12.4	1.75	**1/3.6**

4.3 Arbitrary precision (50 digits)

Function	Mathilda	Mathematica	Maxima	Python	M/best
net per-call latency (ms), loop floor subtracted					
Gamma					
`Gamma`	0.035	0.196	0.714†	**0.030**	1.2
`LogGamma`	**0.035**	0.200	0.338	0.052	**1/1.5**
`PolyGamma0`	0.108	0.123	0.572	**0.098**	1.1
`PolyGamma2`	**0.097**	0.562	1.85	0.727	**1/5.8**
`Beta`	0.099	0.767	1.30	**0.095**	1.0
`Pochhammer`	0.004	**0.002**	0.003	0.064	2.0
Zeta					
`Zeta`	**0.082**	0.837	16.6	0.724	**1/8.8**
`HurwitzZeta`	**0.130**	1.87	–	2.25	**1/14.4**
`LerchPhi`	**0.898**	2.86	–	102	**1/3.2**
`PolyLog`	**0.021**	0.569	–	1.02	**1/27.3**
Bessel/Airy					
`BesselJ`	**0.016**	0.197	2.59	0.036	**1/2.3**
`BesselY`	0.096	0.668	**0.009**	3.26	11.0
`BesselI`	**0.052**	0.092	2.48	0.073	**1/1.4**
`BesselK`	0.104	0.218	**0.006**	3.53	16.2
`AiryAi`	**0.015**	0.142	4.97	0.221	**1/9.7**
`AiryBi`	**0.012**	0.129	4.80	0.124	**1/10.2**
`AiryAiPrime`	**0.013**	0.122	4.79	0.165	**1/9.6**
`AiryBiPrime`	**0.012**	0.127	5.10	0.135	**1/10.3**
`StruveH`	**0.022**	0.727	2.71	0.132	**1/6.1**
Error					
`Erf`	**0.018**	0.399	3.03	0.057	**1/3.1**
`Erfc`	**0.018**	0.298	3.80	0.043	**1/2.3**
`Erfi`	**0.022**	0.329	1.25	0.082	**1/3.8**
`InverseErf`	**0.049**	1.70	5.58	0.591	**1/12.1**
`InverseErfc`	**0.051**	1.63	3.71	0.600	**1/11.7**
`FresnelC`	**0.032**	4.58	39.8	0.086	**1/2.7**
`FresnelS`	**0.023**	0.984	17.8	0.079	**1/3.4**
Elliptic					
`EllipticK`	**0.006**	0.020	2.54	0.029	**1/3.6**
`EllipticE`	**0.013**	0.033	5.16	0.156	**1/2.6**
`EllipticF`	**0.026**	0.106	2.45	0.760	**1/4.1**
`EllipticEinc`	**0.060**	0.133	10.2	2.46	**1/2.2**
`EllipticPi`	0.094	**0.041**	42.2	3.90	2.3
Orthogonal					
`LegendreP`	0.011	0.018	**0.009**	0.035	1.1
`LegendreQ`	**0.043**	0.097	0.412	7.32	**1/2.3**
`HermiteH`	**0.007**	0.013	0.008	0.065	**1/1.2**
`LaguerreL`	0.015	**0.003**	0.006	0.061	4.2
`ChebyshevT`	**0.008**	0.012	0.017	0.039	**1/1.5**
`ChebyshevU`	**0.009**	0.010	0.018	0.043	**1/1.1**
Integral					
`SinIntegral`	**0.018**	0.282	82.5	0.034	**1/2.0**
`CosIntegral`	**0.021**	0.342	92.0	0.048	**1/2.2**
`SinhIntegral`	**0.014**	0.352	7.03	0.118	**1/8.3**
`CoshIntegral`	**0.018**	0.420	7.69	0.118	**1/6.5**
`ExpIntegralEi`	**0.041**	0.174	3.97	0.060	**1/1.5**
`LogIntegral`	**0.038**	0.134	3.31	0.058	**1/1.5**
Hypergeometric					
`Hypergeometric0F1`	0.055	0.087	1.68	**0.033**	1.6
`Hypergeometric1F1`	**0.018**	21.3	2.14	0.048	**1/2.7**

Function	Mathilda	Mathematica	Maxima	Python	M/best
`Hypergeometric2F1`	**0.100**	0.901	3.31	0.384†	**1/9.0**
`HypergeometricPFQ`	0.162	327	2.25	**0.047**	3.5
`HypergeometricU`	**0.002**	0.009	–	0.024	**1/5.2**
`WhittakerM`	**0.021**	0.138	–	0.057	**1/2.7**
`WhittakerW`	**0.020**	0.145	–	0.060	**1/3.0**
`MeijerG`	**0.260**	17.3	–	0.732	**1/2.8**
`MeijerG22`	**0.994**	11.6	–	2.13	**1/2.1**
`MeijerGlog`	**1.51**	41.3	–	7.70	**1/5.1**
Misc					
`Sinc`	**0.005**	0.030	0.037	0.025	**1/5.4**
`ProductLog`	**0.052**	0.165	0.565	0.174	**1/3.2**

5 Speed: vectorized machine-real throughput

Wall time to apply each head to a 10^6-element buffer. This is where Mathilda's packed array / NDArray kernels compete directly with SciPy's vectorized C. Mathematica is too slow to test here: outside a handful of heads with vectorized kernels (`Gamma`, `Erf`, `Sinc`: a few ms), a 10^6-element buffer costs it seconds to minutes (median ≈ 8 s, `StruveH` 334 s), and the three `MeijerG` rows would take hours each. Maxima has no array form and is omitted as well.

Function 10^6-element buffer, ms	Mathilda	Python/SciPy	M/SciPy
`Gamma`	**3.75**	15.3	**1/4.1**
`LogGamma`	**10.4**	21.3	**1/2.0**
`PolyGamma0`	**4.89**	15.9	**1/3.2**
`PolyGamma2`	**14.1**	248	**1/17.6**
`Pochhammer`	**1.43**	7.76	**1/5.4**
`Zeta`	**26.1**	199	**1/7.6**
`PolyLog`	**4.02**	–	–
`BesselJ`	**15.1**	646	**1/42.6**
`BesselY`	**8.15**	1016	**1/124.7**
`BesselI`	**4.85**	386	**1/79.6**
`BesselK`	**6.13**	376	**1/61.4**
`AiryAi`	**3.74**	174	**1/46.5**
`AiryBi`	**3.57**	160	**1/44.7**
`AiryAiPrime`	**4.04**	157	**1/38.9**
`AiryBiPrime`	**3.50**	137	**1/39.1**
`StruveH`	**29.7**	6220	**1/209.5**
`Erf`	**2.97**	14.9	**1/5.0**
`Erfc`	**3.17**	13.9	**1/4.4**
`Erfi`	**3.16**	24.5	**1/7.8**
`InverseErf`	**2.62**	19.3	**1/7.4**
`InverseErfc`	**2.60**	17.3	**1/6.6**
`FresnelC`	**5.27**	34.8	**1/6.6**
`FresnelS`	**5.21**	35.0	**1/6.7**
`EllipticK`	**2.55**	14.5	**1/5.7**
`EllipticE`	**2.68**	14.5	**1/5.4**
`LegendreP`	**1.63**	8.16	**1/5.0**
`LegendreQ`	**3.36**	–	–
`HermiteH`	**1.76**	7.69	**1/4.4**
`LaguerreL`	**1.81**	10.3	**1/5.7**
`ChebyshevT`	**1.70**	5.11	**1/3.0**
`ChebyshevU`	**1.74**	4.56	**1/2.6**
`SinIntegral`	**4.96**	28.2	**1/5.7**
`CosIntegral`	**6.57**	28.1	**1/4.3**

Function 10^6-element buffer, ms	Mathilda	Python/SciPy	M/SciPy
SinhIntegral	**4.04**	107	**1/26.5**
CoshIntegral	**4.46**	106	**1/23.7**
ExpIntegralEi	**3.97**	155	**1/39.0**
ExpIntegralE	**19.9**	224	**1/11.3**
LogIntegral	**5.86**	–	–
Hypergeometric0F1	**83.2**	233	**1/2.8**
Hypergeometric1F1	**9.84**	23.6	**1/2.4**
Hypergeometric2F1	**122**	146	**1/1.2**
HypergeometricPFQ	**163**	–	–
HypergeometricU	**4.54**	33.9	**1/7.5**
WhittakerM	**14.3**	–	–
WhittakerW	**15.3**	–	–
MeijerG	**1036**	–	–
MeijerG22	**229**	–	–
MeijerGlog	**320**	–	–
Sinc	**2.71**	–	–
ProductLog	**18.1**	230	**1/12.7**

6 Exact arithmetic

The integer/rational-valued heads, compared by exact value (agreement is exact string equality at the smallest size) and timed at growing argument sizes. All engines that compute a given head returned byte-identical exact values on the 7 functions where at least two systems evaluate it.

Function	n	Mathilda	Mathematica	Maxima	Python
time (ms); value agreement checked at the smallest n					
Fibonacci	500	0.044	1.34	0.621	0.026
	5000	0.021	0.043	0.047	0.115
	50000	0.113	0.112	0.416	–
LucasL	500	0.006	0.022	0.047	0.436
	5000	0.010	0.023	0.043	0.164
	50000	0.083	0.053	0.294	–
BernoulliB	50	0.261	0.270	0.446	0.017
	200	5.05	2.72	17.0	0.490
	500	48.5	18.3	146	0.972
EulerE	20	0.005	0.043	0.050	0.011
	50	0.008	0.660	0.216	0.781
	100	0.066	1.99	0.686	2.75
HarmonicNumber	100	0.283	0.572	–	0.168
	1000	6.50	0.160	–	2.44
BarnesG	20	0.003	1.41	–	–
	60	0.012	0.218	–	–
Hyperfactorial	20	0.004	0.097	–	–
	60	0.040	0.178	–	–

7 Discussion

Where Mathilda stands. The medians over the sweep, as net latency against the fastest other engine: machine real 8.2× faster, complex 9.9× faster, arbitrary precision 2.7× faster. Mathilda has the lowest net latency in *every* machine-real and machine-complex cell, and its 10^6-element buffers beat SciPy for every head SciPy implements.

How the machine regimes were closed (v0.436–v0.444). The first version of this report (v0.433) had Mathilda a median 12× slower for machine reals and 9× for complex arguments. Part of that was the measurement itself: 17-digit literals were read as arbitrary precision, and the `Do`/`Part`/`Quiet` loop floor was charged to every call. Both are fixed in the method above. The rest was real, and had three causes.

- **Scalar paths bypassed the machine kernels.** Most scalar builtins recomputed in MPFR, or rebuilt and re-evaluated an expression, even where an `sf_machine` double kernel already existed. Every special-function builtin now goes first through one shared machine gate (`sf_machine_scalar_args`) to the same raw-double kernel the packed, NDArray and `Compile[]` surfaces use, so the five surfaces agree bit for bit.
- **No native complex kernels.** The `ncpx`-style MPFR pairs are replaced, for machine input, by raw double re/im kernels for every head:
  - Faddeeva $w(z)$ for the error family and Fresnel
  - Amos-style I/K reduction with Taylor continuation for Bessel and Airy
  - Euler–Maclaurin and inversion maps for Zeta, HurwitzZeta, LerchPhi and PolyLog
  - tiled Taylor/Stirling for $\ln\Gamma$ and ψ
  - Carlson's integrals in complex arithmetic for the elliptic integrals
  - transformation-selected series and ODE continuation for the hypergeometric family
- **Kernel algorithms.** Real kernels that lost to SciPy were rebuilt as fitted rational, Chebyshev and auxiliary-function approximations. Their coefficients are regenerated from mpmath by scripts under `tools/`.

Two evaluator costs were also cut. The confirming step after a builtin returns a numeric or canonical-complex atom is gone, and so is the packed-array gate scan for calls with no buffer argument. Accuracy rose with speed: the agreement table is at or above its v0.433 values, and every kernel was gated against mpmath (median $\lesssim 1$ ulp) before it was wired.

MeijerG (v0.445, v0.446). Three rows cover the Meijer G-function: a two-term $G^{2,1}_{2,3}$ (`MeijerG`), a $G^{2,2}_{2,2}$ whose Slater expansion switches from z to $1/z$ at the unit circle (`MeijerG22`), and the documented log case of integer-spaced b's (`MeijerGlog`). SciPy has no G-function and Maxima no numeric one, so the Python column is mpmath's `meijerg` and Maxima is absent. Mathilda is the fastest engine in all nine cells: against mpmath 32×, 128× and 539× (machine real), 42×, 245×, 469× (complex) and 2.8×, 2.1×, 5.1× (50 digits), and Mathematica is slower still in every cell (12–6500×). The machine path is one Slater plan per parameter set (the z-independent $\pi/\sin$ and Γ products) and one pFq kernel call per term; a cancelling sum is handed to the arb engine. On a 10^6-element buffer the plan is built once for the whole buffer, not once per call, so the two non-log rows run at 1.0 and 0.23 μs per element. SciPy has no G-function, so these throughput rows have no comparison column. The 50-digit `MeijerG22` cell first came out level with mpmath (2.03 vs 2.07 ms) because N of the exactly reduced ${}_2F_1$ pair summed each series directly; routing arbitrary-precision `Hypergeometric2F1` through arb's transformation-based kernel fixed that, and took the `Hypergeometric2F1` AP cell from 0.62 to 0.10 ms (3.8× ahead of mpmath).

The MeijerG log case (v0.446). At v0.445 the log case had no machine form: every point went to the arb engine, which perturbs the colliding b's and compares two perturbation sizes. That cost 456 μs per call and 119 s per 10^6 buffer. It is now summed on the machine path as the residue series of the Mellin–Barnes integrand, $G = \sum_k w^{\beta+k} \sum_j c_{kj} \log^j w$, one series per integer-spaced class of b's with base β. The c_{kj} do not depend on z, so the plan tabulates them once: a Laurent expansion from arb, then the exact rational step $F(s+1) = F(s)R(s)$ in double-double. A point

is summed in double, re-summed in double-double when it measurably loses more than 3 bits, and handed to arb only past 45. MeijerGlog now costs 2.99 μs (real), 7.6 μs (complex) and 320 ms per 10^6 buffer, 85–370× faster than v0.445. Its agreement figure (12.4 digits) is set by Mathematica, whose machine log case is off by 3×10^{-12} at $z = 5.5$; Mathilda and mpmath match 40-digit values to $\approx 10^{-16}$. The MeijerG rows were re-run at v0.446 and merged with the v0.445 hypergeometric run and the v0.444 run of the other heads (results/2026-10-17-final-v0446.json).

Notes on the flagged disagreements. The recurring genuine differences are: Maxima's expintegral_chi returns a value with a spurious $i\pi$ imaginary part on the positive real axis (a branch convention); Maxima leaves hurwitz_zeta, polylog, HypergeometricU, WhittakerM/W and BarnesG symbolic (no numeric kernel, shown as absent, not as a disagreement); and branch-cut choices for multivalued heads differ between systems off the principal domain.

8 Reproducing

```
cd <mathilda-repo>
make -j                        # build ./Mathilda
benchmarks/.venv/bin/python benchmarks/97-special-functions/run_shootout.py --ap-dps 50
benchmarks/.venv/bin/python benchmarks/97-special-functions/report.py [results/<file>.json]
cd benchmarks/97-special-functions && latexmk -pdf special_function_performance.tex
```

The driver writes results/<date>.json; report.py regenerates every tab_*.tex fragment and macros.tex from it. Engine binaries are found via MATHILDA_BIN, WOLFRAMSCRIPT, MAXIMA_BIN, and the benchmark venv.